MoTE Ulysses 接入与验证

2026-11-06 · 有边界的训练接入 · 本报告更新 11 月 5 日可行性审查的实现状态

已完成的接入

Qwen3 和采用 Llama3.1 RoPE 的全注意力模型已接入官方 DeepSpeed-Ulysses，复用 FitMoTNTrainer 完成训练、梯度累积、检查点与恢复。MoTE 的容量裁剪和 QuantumGate 保持原算法；四进程 SP=2、DP=2 的训练更新对照通过。当前验证环境为 Linux / CPU / Gloo、随机小模型、FP32、序列长度 16。实际大模型的 16K～32K GPU 显存和吞吐仍需硬件验收。

新增入口是 python -m fitmotn.train.ulysses。它复用现有模型恢复、优化器分组与检查点事务工具；既有阶段控制器继续使用原入口。没有修改 gate.py、quantum_gate.py、ADTN.py、model.py、sparse_mixt.py、patching.py。

计算语义与实现

注意力的 all-to-all 和数据序列切分直接调用 DeepSpeed 0.19.9 的 UlyssesSPAttentionHF、UlyssesSPDataLoaderAdapter。位置编号在切分前生成；标签与监督权重都先在完整序列上对齐下一 token 目标，再切分，保留跨分片的监督。

路由器在本卡计算 logits，适配层使用 PyTorch 自动求导集合通信汇总 token × expert 分数，恢复 batch 优先的全局 token 顺序。原有 gate 在这些分数上执行 top-k、容量选择、归一化和 aux，再将路由结果切回本卡。FFN 隐藏状态和专家计算仍按序列分片，不需要为路由复制整段隐藏状态。全局路由分数的内存与通信开销随序列长度和专家数量增长。

监督损失在 SP 组内汇总有效权重分母，本地损失乘以 SP 大小，抵消 DDP 的梯度平均。DP 组之间平均各逻辑微批的损失；梯度累积继续由 Trainer 处理。全局 aux 在各卡保持一致，DDP 平均后计入一次。默认 aux-weight=0，延续原 SFT 的 CE 目标；显式设置 1 才加入各 gate 已带系数的 l_aux 之和。

数据采用带内容 SHA256 的可索引 JSONL。DistributedSampler 在所有进程间分发不同样本，再由官方适配器将 SP 组内各来源微批依次切分。有效全局 batch = 每卡微批 × DP 大小 × 梯度累积；SP 组内的卡共同处理同一批。各 epoch 的尾部不足完整分组时丢弃，与此入口的 drop_last 约定一致。

检查点由所有 rank 写入暂存目录，确认每个 rank 的 RNG 文件齐全后，由 rank 0 校验并原子发布。仅 rank 0 写 MoTE 元数据；恢复核对数据指纹、SP/DP 拓扑、微批、累积步数和训练设置，并按确定性顺序重放跳过已完成批次。输出目录需由所有进程共享。最终样本吞吐按逻辑样本计数，避免将 SP 分片重复计算。

实测结果

四进程正式接入测试使用原生 Trainer.train、SGD momentum=0.9、实际 Gloo 集合通信。Qwen3/Llama 为 2 层、hidden=32、intermediate=64、4 个注意力头和 2 个 KV 头；MoTE 为 4 专家、top-k=2、capacity_factor=0.5、min_capacity=1，并启用全局专家。QuantumGate 使用 rank=4、depth=1、block_size=2。测试包含加权监督、某个 SP 分片没有有效标签、batch=2 和累积 2 步。

模型	连续训练相对完整序列参考的最大参数误差	中断恢复后相对连续训练
Qwen3 dense	7.45e-9	0
Llama3.1 RoPE 配置	3.73e-9	0
Qwen3-MoTE，容量裁剪与 aux	5.96e-8	0
Qwen3-QuantumGate，容量裁剪与 aux	2.98e-8	0

前三项执行 3 次优化器更新，QuantumGate 执行 5 次并跨越 epoch 边界；每项均从 checkpoint-1 重启对照。所有 rank 的结果与检查点校验通过。另以实际 torchrun 入口，从本地小型 QuantumGate 检查点恢复结构，完成 AdamW 训练及中途恢复，核对模型、优化器和调度器状态。相关回归为 250 passed、1 skipped；一条原有初始化回退警告保留。

证据位于 research/ulysses_integration_20261106/。测试代码位于 tests/distributed_ulysses.py；独立 CPU 入口检查使用 tests/ulysses_cpu_entrypoint.py，仅关闭需要 C++ 编译器的可选 SHM 优化，保留官方 Ulysses 与真实 Gloo 通信。

失败分析与当前边界

四进程初测发现 PyTorch 旧 distributed.nn.functional 的 Gloo 反向路径将子组内编号误当作全局编号，导致不含 rank 0 的 SP 组失败。适配改用 PyTorch 2.10.0 官方 functional 自动求导集合通信，按进程组名称执行 gather / reduce-scatter，四进程复测通过。这也是固定运行库版本的原因之一。

实际 CPU torchrun 恢复时，Accelerate 的 cpu:0 设备值不能用于 PyTorch storage 的反序列化。接入层对 CPU 优化器和调度器恢复明确使用 cpu，GPU 路径继续复用 Trainer。原始失败日志保留在本地，修复后的验收结果纳入发布证据。

首版限定全注意力 Qwen3 / Llama、SDPA、固定长度、无 padding、无文档边界掩码。允许 labels=-100 的监督掩码。Gemma3 窗口、Qwen3.5 混合递归注意力、非零 dropout / router jitter、梯度重计算、ZeRO / FSDP、KD、旧阶段控制器的统计与中途评测均会明确拒绝，避免训练语义静默改变。Megatron-SP / CP 的自定义 MoTE 模块适配尚待实现。

当前使用 DDP，模型参数与优化器状态仍在各卡复制；仅靠此接入不能推定大模型可训练到 32K。下一步应先在 Linux NVIDIA 两卡测量短序列正确性和显存，再逐级增加到 4K、8K、16K、32K，结合重计算与词表损失分块继续适配。Windows 异构卡组和 BW1000 DCU 需要分别验证通信与算子后端；本轮没有它们的 GPU 运行证据。

使用入口

在独立 Linux 环境安装与硬件匹配的 PyTorch 2.10.0，再安装项目的 ulysses 可选依赖。固定版本为 Transformers 4.57.6、Accelerate 1.15.0、DeepSpeed 0.19.9。请保持活动实验的冻结环境不变。

准备 JSONL，每行包含等长 input_ids、labels，可选 loss_weights。示意记录为 {"input_ids":[1,2,3,4],"labels":[-100,2,3,4]}。实际长度必须等于 seq-length，且可被 sp-size 整除；注意力头数也必须支持对应并行度。不要把已有 padding 或需要隔离的文档掩码删掉以通过校验，应先调整数据构建方式。

两卡命令示例：torchrun --standalone --nproc-per-node=2 -m fitmotn.train.ulysses --model /models/qwen3-mote --data /data/train-tokens.jsonl --output /runs/qwen3-sp --sp-size 2 --seq-length 4096 --steps 100 --batch-size 1 --grad-accum 4 --save-steps 20。

此命令是待在目标 GPU 环境验收的使用示例。默认 FP32；支持显式选择 dtype。FitMoTN 检查点默认仅训练已替换 FFN；普通 HF 模型训练全部参数。恢复使用相同设置并添加 --resume /runs/qwen3-sp/checkpoint-20。检查点包含逐 rank 状态与数据指纹，改变拓扑、数据或已记录的训练协议会被拒绝。

整理与实验保护

公共文件哈希、序列化、分位数统计、日志句柄和显存诊断已归并；CE/KD 目标与延迟统计从大 Trainer 模块分离，旧导入名仍可用。整理的 8 个函数与原始 AST 一致。此类重构减少重复代码，没有以新的模型分数或吞吐成绩作结论。

活动实验 D:/AI/MOTE 继续保持冻结提交 860426ce3df30f5c2a74a030a43f22e3a76247c6。外部运行时归档及原始源文件的 117 文件校验通过。本次变更均在独立发布工作树完成。